Selected Topics in AI Accelerators Hardware Design

ELCT918

Verilog Refresher Assignment

Crack the Cipher Chip — Black-Box Reverse Engineering in Verilog

Report

Submitted to	Dr. Eman Azab · MSc Eng. Nour ElShahawy
Student name	**Ahmed Mohamed Ahmed**
Student ID	**58-24418**
GitHub repository	https://github.com/ahmedSheta26/ELCT918
Date	October 2026

Summary of Results

The chip is a small stream cipher. For every enabled clock cycle it adds a position-dependent key to the 4-bit plaintext nibble, counts the nibbles, and remembers the last four ciphertext nibbles. `flag` is raised when those four nibbles spell a secret word. Everything below was found from simulation experiments only.

Item	Result
`dout[7:4]`	Counter of enabled nibbles since reset (4 bits, wraps F → 0)
`dout[3:0]`	Ciphertext = plaintext + key (mod 16); key sequence 6, 7, 4, 5 repeating every 4 nibbles (= 6 XOR position)
Internal state	4-bit counter + the last 4 ciphertext nibbles (16 bits)
`flag`	High while the last four ciphertext nibbles are C, A, F, E ("CAFE"); held while `en`=0
Password (plaintext)	**6 3 B 9** (ciphertext C A F E)
Self-check	PASS — 0 mismatches in 40,005 random cycles

1. Experiments

All experiments use the testbench template from the assignment: the task `step(en, din)` applies one input for one clock cycle and prints the outputs just after the rising edge. In the final testbench the same stimulus drives both `mystery` and my `actual` module, so every log line shows both. The two halves are always identical (this is the self-check of Section 3), so here the `mystery:` half is read as “what the chip did”. Simulator: questasim. Time is printed in picoseconds (46000 = 46 ns); the clock period is 10 ns.

Experiment 0 – Reset

Tested	`rst_n` held low for two clock edges from power-up, then released.
Observed	`after reset: dout=00 flag=0`
Concluded	Reset clears the 8 output bits and `flag`. Together with Experiment 4 this shows that reset starts a new message.

```
after reset: mystery dout=00 flag=0 | actual dout=00 flag=0
```

Experiment 1 – What happens when en is low?

Tested	`en`=0 with `din` = 5 and then 9 for one clock each, right after reset.
Observed	`dout` stays 00000000 and `flag` stays 0 in both cycles.
Concluded	With `en`=0 the chip ignores `din` completely: nothing is encrypted, counted or stored.

```
t=26000 rst_n=1 en=0 din=5 -> mystery: dout=00000000 flag=0 | actual: dout=00000000 flag=0
t=36000 rst_n=1 en=0 din=9 -> mystery: dout=00000000 flag=0 | actual: dout=00000000 flag=0
```

Experiment 2 – The same plaintext many times

Tested	`din`=1 with `en`=1 for 15 cycles (the hint in the assignment), then one cycle with `en`=0, then one more cycle with `en`=1.
Observed	The upper nibble of `dout` counts 1, 2, 3 … F. The lower nibble does not stay constant even though `din` is always 1; it repeats 7, 8, 5, 6, 7, 8, 5, 6 … (period 4). During the pause (`en`=0) `dout` holds F5. On the next enabled cycle `dout` = 06: the counter wrapped from F to 0 and the ciphertext is 6.
Concluded	(a) `dout[7:4]` is a 4-bit counter of enabled nibbles that wraps after 16. (b) The ciphertext depends on a key that changes with every nibble and repeats every 4 nibbles. (c) The key sequence is not disturbed by the wrap, because 16 is a multiple of 4: the 16th nibble gets the 4th key, and 1 + 5 = 6 matches.

```
t=46000 rst_n=1 en=1 din=1 -> mystery: dout=00010111 flag=0 | actual: dout=00010111 flag=0
t=56000 rst_n=1 en=1 din=1 -> mystery: dout=00101000 flag=0 | actual: dout=00101000 flag=0
t=66000 rst_n=1 en=1 din=1 -> mystery: dout=00110101 flag=0 | actual: dout=00110101 flag=0
t=76000 rst_n=1 en=1 din=1 -> mystery: dout=01000110 flag=0 | actual: dout=01000110 flag=0
t=86000 rst_n=1 en=1 din=1 -> mystery: dout=01010111 flag=0 | actual: dout=01010111 flag=0
t=96000 rst_n=1 en=1 din=1 -> mystery: dout=01101000 flag=0 | actual: dout=01101000 flag=0
```

```
t=106000 rst_n=1 en=1 din=1 -> mystery: dout=01110101 flag=0 | actual: dout=01110101 flag=0
t=116000 rst_n=1 en=1 din=1 -> mystery: dout=10000110 flag=0 | actual: dout=10000110 flag=0
t=126000 rst_n=1 en=1 din=1 -> mystery: dout=10010111 flag=0 | actual: dout=10010111 flag=0
t=136000 rst_n=1 en=1 din=1 -> mystery: dout=10101000 flag=0 | actual: dout=10101000 flag=0
t=146000 rst_n=1 en=1 din=1 -> mystery: dout=10110101 flag=0 | actual: dout=10110101 flag=0
t=156000 rst_n=1 en=1 din=1 -> mystery: dout=11000110 flag=0 | actual: dout=11000110 flag=0
t=166000 rst_n=1 en=1 din=1 -> mystery: dout=11010111 flag=0 | actual: dout=11010111 flag=0
t=176000 rst_n=1 en=1 din=1 -> mystery: dout=11101000 flag=0 | actual: dout=11101000 flag=0
t=186000 rst_n=1 en=1 din=1 -> mystery: dout=11110101 flag=0 | actual: dout=11110101 flag=0
t=196000 rst_n=1 en=0 din=1 -> mystery: dout=11110101 flag=0 | actual: dout=11110101 flag=0
t=206000 rst_n=1 en=1 din=1 -> mystery: dout=00000110 flag=0 | actual: dout=00000110 flag=0
```

Experiment 3 – The complete cipher table

Tested	For each plaintext nibble 0 … F: reset, then send that nibble four times (positions 0 … 3) and record the lower nibble of `dout`.
Observed	Each row of the table is the plaintext sequence shifted by a constant, wrapping from F to 0. The shifts are 6, 7, 4, 5 for positions 0, 1, 2, 3.
Concluded	Ciphertext = (plaintext + K[position]) mod 16 with K = 6, 7, 4, 5. Because the result wraps, the operation is a 4-bit addition (an XOR key would permute bits instead of shifting). K can be written as 6 XOR position (0110 XOR 00/01/10/11), and only the two low bits of the position matter.

```
plain : 0 1 2 3 4 5 6 7 8 9 a b c d e f
pos 0 : 6 7 8 9 a b c d e f 0 1 2 3 4 5    (key = cipher - plain = 6)
pos 1 : 7 8 9 a b c d e f 0 1 2 3 4 5 6    (key = cipher - plain = 7)
pos 2 : 4 5 6 7 8 9 a b c d e f 0 1 2 3    (key = cipher - plain = 4)
pos 3 : 5 6 7 8 9 a b c d e f 0 1 2 3 4    (key = cipher - plain = 5)
```

Experiment 4 – Reset in the middle of a message

Tested	Reset, three nibbles of 3, then a cycle with `rst_n`=0 **and** `en`=1, then a nibble 3 again.
Observed	After the first nibble `dout` = 19. After the reset cycle `dout` = 00. The following nibble gives 19 again, exactly as the first nibble of the message did.
Concluded	Reset has priority over `en`. It resets the counter and the key position (and the remembered ciphertext, see Experiment 6), so the next nibble is again nibble number 1 of a new message.

```
t=1016000 rst_n=0 en=0 din=0 -> mystery: dout=00000000 flag=0 | actual: dout=00000000 flag=0
t=1026000 rst_n=1 en=1 din=3 -> mystery: dout=00011001 flag=0 | actual: dout=00011001 flag=0
t=1036000 rst_n=1 en=1 din=3 -> mystery: dout=00101010 flag=0 | actual: dout=00101010 flag=0
t=1046000 rst_n=1 en=1 din=3 -> mystery: dout=00110111 flag=0 | actual: dout=00110111 flag=0
t=1056000 rst_n=0 en=1 din=3 -> mystery: dout=00000000 flag=0 | actual: dout=00000000 flag=0
t=1066000 rst_n=1 en=1 din=3 -> mystery: dout=00011001 flag=0 | actual: dout=00011001 flag=0
```

Experiment 5 – Finding the password (exhaustive search)

Tested	The assignment says `flag` depends on the last four ciphertext nibbles. A temporary testbench tried **all 65,536** four-nibble plaintext messages: reset, send the four nibbles with `en`=1, look at `flag` after each nibble. (Code in Appendix C.)
Observed	Exactly one message raised `flag`: `flag HIGH after message 63b9 (dout=4e)`. The summary line was `hits=1 early_flags=0`, so `flag` was never high after the first three nibbles of any message.

Concluded	The password is **6 3 B 9**. Using the key table of Experiment 3 its ciphertext is C, A, F, E (Section 4).

Experiment 6 – How does flag behave?

Tested	(a) Send the password and then hold `en`=0 for two cycles, then send one more nibble. (b) A near miss: 6 3 B **8**. (c) CAFE ciphertext at a different alignment: two junk nibbles first, then plaintexts chosen (using the key table) so that the ciphertext is C, A, F, E.
Observed	(a) `flag` rises together with `dout` on the 4th nibble, stays high while `en`=0, and drops with the next enabled nibble. (b) `flag` stays 0. (c) `flag` is also 1 after the 6th nibble of the message.
Concluded	`flag` does not depend on the plaintext or on the position: it is high exactly when the last four ciphertext nibbles (oldest to newest) are C, A, F, E. It is computed from registered state, so it changes on the same clock edge as `dout` and is held while `en`=0. The chip therefore remembers the last four ciphertext nibbles, which clear on reset.

(a) Password and hold:

```
t=1076000 rst_n=0 en=0 din=0 -> mystery: dout=00000000 flag=0 | actual: dout=00000000 flag=0
t=1086000 rst_n=1 en=1 din=6 -> mystery: dout=00011100 flag=0 | actual: dout=00011100 flag=0
t=1096000 rst_n=1 en=1 din=3 -> mystery: dout=00101010 flag=0 | actual: dout=00101010 flag=0
t=1106000 rst_n=1 en=1 din=b -> mystery: dout=00111111 flag=0 | actual: dout=00111111 flag=0
t=1116000 rst_n=1 en=1 din=9 -> mystery: dout=01001110 flag=1 | actual: dout=01001110 flag=1
>>> flag went HIGH on the 4th nibble (dout=4e)
t=1126000 rst_n=1 en=0 din=0 -> mystery: dout=01001110 flag=1 | actual: dout=01001110 flag=1
t=1136000 rst_n=1 en=0 din=f -> mystery: dout=01001110 flag=1 | actual: dout=01001110 flag=1
t=1146000 rst_n=1 en=1 din=0 -> mystery: dout=01010110 flag=0 | actual: dout=01010110 flag=0
```

(b) Near miss:

```
t=1156000 rst_n=0 en=0 din=0 -> mystery: dout=00000000 flag=0 | actual: dout=00000000 flag=0
t=1166000 rst_n=1 en=1 din=6 -> mystery: dout=00011100 flag=0 | actual: dout=00011100 flag=0
t=1176000 rst_n=1 en=1 din=3 -> mystery: dout=00101010 flag=0 | actual: dout=00101010 flag=0
t=1186000 rst_n=1 en=1 din=b -> mystery: dout=00111111 flag=0 | actual: dout=00111111 flag=0
t=1196000 rst_n=1 en=1 din=8 -> mystery: dout=01001101 flag=0 | actual: dout=01001101 flag=0
```

(c) CAFE at another alignment (plaintexts 8, 5, 9, 7 at positions 2 … 5):

```
t=1206000 rst_n=0 en=0 din=0 -> mystery: dout=00000000 flag=0 | actual: dout=00000000 flag=0
t=1216000 rst_n=1 en=1 din=0 -> mystery: dout=00010110 flag=0 | actual: dout=00010110 flag=0
t=1226000 rst_n=1 en=1 din=0 -> mystery: dout=00100111 flag=0 | actual: dout=00100111 flag=0
t=1236000 rst_n=1 en=1 din=8 -> mystery: dout=00111100 flag=0 | actual: dout=00111100 flag=0
t=1246000 rst_n=1 en=1 din=5 -> mystery: dout=01001010 flag=0 | actual: dout=01001010 flag=0
t=1256000 rst_n=1 en=1 din=9 -> mystery: dout=01011111 flag=0 | actual: dout=01011111 flag=0
t=1266000 rst_n=1 en=1 din=7 -> mystery: dout=01101110 flag=1 | actual: dout=01101110 flag=1
```

The testbench also checks the “random” cases of Section 3, in which CAFE and near-CAFE sequences are injected at all four alignments. They are compared against the chip cycle by cycle, which is how I convinced myself that there is no other hidden dependence (for example on earlier plaintext nibbles).

2. How the Chip Works

2.1 Internal state

State	Width	Meaning
`count`	4 bits	Number of nibbles encrypted since reset, modulo 16. It is shown directly on `dout[7:4]`. Its two low bits select the key.
`history`	16 bits	The last four ciphertext nibbles. `history[3:0]` is the newest (shown on `dout[3:0]`), `history[15:12]` is the oldest.

2.2 What happens on each rising clock edge

Condition	`count`	`history`	Outputs
`rst_n`=0 (wins over `en`)	→ 0	→ 0000	`dout`=00, `flag`=0
`rst_n`=1, `en`=0	unchanged	unchanged	unchanged (`din` ignored)
`rst_n`=1, `en`=1	+ 1 (wraps F → 0)	shift left 4 bits, new ciphertext in at the bottom	`dout` and `flag` show the new state

For an enabled cycle the new ciphertext nibble is computed from the **old** counter value:

1. `key = 4'h6 XOR {2'b00, count[1:0]}` — the key for this position (6, 7, 4, 5).
2. `cipher = din + key` — a 4-bit addition that wraps modulo 16.
3. `history <= {history[11:0], cipher}` — the older three nibbles move up and the new one enters.
4. `count <= count + 1` — the next nibble will use the next key.

2.3 How dout and flag are formed

- `dout = {count, history[3:0]}` — the upper nibble is the (already incremented) counter and the lower nibble is the newest ciphertext. Both come straight from flip-flops, so `dout` changes just after the clock edge.
- `flag = (history == 16'hCAFE)` — a comparison of registered state with the secret word, with the oldest ciphertext nibble (C) compared first. Because it depends only on registers it also changes just after the clock edge and stays as it is while `en`=0.

2.4 Worked example

Message 6, 3, B, 9 after reset. Because `dout` shows the counter after the edge, the counter in the table is already incremented.

Nibble	Plaintext	Key	Ciphertext	`dout`	`flag`
1 (pos 0)	6	6	6 + 6 = C	0x1C	0
2 (pos 1)	3	7	3 + 7 = A	0x2A	0
3 (pos 2)	B	4	B + 4 = F	0x3F	0
4 (pos 3)	9	5	9 + 5 = E	0x4E	1 (C A F E)

3. My Solution and Self-Check

3.1 The RTL (actual)

The module `actual` has exactly the same ports as `mystery` and is written as clean RTL: a named counter, a named 16-bit ciphertext history, the key and the cipher as separate wires, and one clocked process with the three cases of Section 2.2. It does not copy or translate the netlist. Every line is commented.

- `count` and `history` are the only registers (20 flip-flops in total).
- The key is `BASE_KEY ^ count[1:0]`; the cipher is `din + key`, a plain 4-bit adder.
- `dout` and `flag` are continuous assignments from the registers, so they match the chip's timing exactly.
- The secret word is the named constant `SECRET = 16'hCAFE`.

3.2 The self-checking testbench

`testbench.sv` instantiates `mystery` (`bb`) and `actual` (`dut`) side by side and drives them with the same inputs through one task, `drive(rst_n, en, din)`. After every clock edge it compares `dout` and `flag` of both with `!==` (so an X also counts as a mismatch), counts mismatches, prints the first ten with their cycle number, and finishes with PASS or FAIL. It contains:

- the directed experiments of Section 1 and the password test of Section 4;
- a random run of at least 40,000 cycles using `$random` with a fixed seed, in four randomly chosen modes: 50/50 `en` with resets about every 64 cycles; long `en`=1 bursts so the counter wraps many times; mostly `en`=0; and rare sequences in which CAFE (or CAFE with one nibble changed) is produced at a random alignment, sometimes after a reset;
- coverage counters (flag-high cycles and counter wraps) so that the run can be seen to exercise the rare cases.

3.3 Self-check result

```
cycles compared : 40005
flag-high cycles: 2065   counter wraps: 1091
PASS: actual matches mystery on every cycle (0 mismatches)
```

Run	Cycles compared	Mismatches
Submitted testbench (seed C0FFEE)	40,005	0

4. The Password

4.1 How it was derived (without looking at the netlist)

1. **What flag depends on.** Experiments 2 and 3 show that the lower nibble of `dout` is the ciphertext and that the same plaintext gives different ciphertexts at different positions. The assignment states that `flag` is high when the last four ciphertext nibbles are a famous 4-letter hex word, so the password must be four plaintext nibbles whose ciphertexts form that word.
2. **Exhaustive search on the chip.** The password is only 16 bits long, so I tried every one of the 65,536 messages directly on `mystery` (Experiment 5). Exactly one raised `flag`: **6 3 B 9**.
3. **Cross-check with the cipher model.** Subtracting the keys 6, 7, 4, 5 from the word CAFE gives C−6 = 6, A−7 = 3, F−4 = B, E−5 = 9, i.e. the same message. Other famous words give different messages that do not raise `flag`: BEEF would be 5 7 A A and DEAD would be 7 7 6 8, and neither was found by the search.
4. **Unique answer.** Because each position is a one-to-one addition modulo 16, every four-nibble ciphertext corresponds to exactly one plaintext message, so there is no other message for CAFE. This agrees with the search result `hits=1`.
5. **Check of the sliding window.** Experiment 6(c) shows that CAFE ciphertext at any other alignment also raises `flag`, so the chip compares the last four ciphertext nibbles and not the plaintext.

Position	Plaintext	Key	Ciphertext (plain + key)	`dout` after the edge
0	6	6	C	0x1C
1	3	7	A	0x2A
2	B	4	F	0x3F
3	9	5	E	0x4E → `flag`=1

4.2 Simulation output with flag going high

The password test of the testbench resets the chip and applies 6, 3, B, 9 on four consecutive enabled cycles. `flag` rises on the fourth (`dout`=4E), stays high while `en`=0, and drops with the next nibble:

```
t=1076000 rst_n=0 en=0 din=0 -> mystery: dout=00000000 flag=0 | actual: dout=00000000 flag=0
t=1086000 rst_n=1 en=1 din=6 -> mystery: dout=00011100 flag=0 | actual: dout=00011100 flag=0
t=1096000 rst_n=1 en=1 din=3 -> mystery: dout=00101010 flag=0 | actual: dout=00101010 flag=0
t=1106000 rst_n=1 en=1 din=b -> mystery: dout=00111111 flag=0 | actual: dout=00111111 flag=0
t=1116000 rst_n=1 en=1 din=9 -> mystery: dout=01001110 flag=1 | actual: dout=01001110 flag=1
>>> flag went HIGH on the 4th nibble (dout=4e)
t=1126000 rst_n=1 en=0 din=0 -> mystery: dout=01001110 flag=1 | actual: dout=01001110 flag=1
t=1136000 rst_n=1 en=0 din=f -> mystery: dout=01001110 flag=1 | actual: dout=01001110 flag=1
t=1146000 rst_n=1 en=1 din=0 -> mystery: dout=01010110 flag=0 | actual: dout=01010110 flag=0
```

5. GitHub Repository and Files

Item	Content
Repository	**https://github.com/ahmedSheta26/ELCT918**
Folder	`Verilog assignment`
`design.sv`	The unchanged `mystery` module followed by my module `actual`
`testbench.sv`	Experiments, self-check against the chip, password test
`wave.do`	A do file to compile, run and add signals to the waveform
`source.f`	File containing the .v and .sv files used in simulation
`README.md`	Explaining the functionality